Unit03-Student-Textbook

Unit 03 — Testing

Software Quality · Computer Engineering · USJ 2026

> *"Testing can only show the presence of bugs, never their absence."*
> — Edsger W. Dijkstra

Table of Contents

---

1. What Testing Is — and What It Is Not

Testing is the act of using a software product with the explicit goal of detecting failures. That definition is deliberately narrow and deliberately active: you are not a passive observer hoping the system happens to behave, you are an adversary probing it for weaknesses. Understanding that definition correctly is the first step toward being a useful tester.

What testing is **not** is equally important. Testing does not prove that software is correct. This surprises many students because it sounds like the whole point. But the mathematics are against you. Consider a function that takes a single 32-bit integer as input. To test every possible value exhaustively you would need to execute more than four billion test cases — and that is a function with a single parameter. Real software has dozens of parameters, each with enormous value ranges, operating in sequences and combinations that produce a space of inputs that is effectively infinite. Exhaustive testing is not a practical ambition; it is a theoretical impossibility. As Dijkstra observed, testing can only reveal the presence of defects, never their absence. The absence of failure in your test suite does not mean the

software is correct; it means your test cases did not trigger the latent defects that may still be lurking.

Testing also does not fix defects. This is a process boundary, not a pedantic one. The tester's job is to surface evidence of failure and report it with enough fidelity that a developer can reproduce and diagnose it. Conflating these roles leads to confusion about accountability.

The Bug → Error → Failure Chain

The industry distinguishes three related concepts that are often used interchangeably in casual conversation but mean very different things technically.

A **bug** (also called a fault or defect) is a mistake in the source code — a wrong statement, a missing condition, an incorrect constant. A bug exists in the static artefact. It may never execute.

An **error** is the manifestation of a bug during execution. When the program runs and the incorrect code path executes, the program's internal state diverges from what it should be. The error lives in memory — it is a corrupted intermediate state that the user cannot yet see.

A **failure** is the observable consequence: the moment the corrupted internal state reaches the surface and the system behaves differently from its specification. The user sees the wrong result, the system crashes, the file is truncated.

A concrete example makes the chain tangible. Suppose you are implementing an income tax calculator. The specification says that income above 30,000 € is taxed at 37%. You accidentally write `0.037` instead of `0.37` in the code. That incorrect constant is the **bug** — it exists in the source file whether or not you run the program. When a user submits income of 50,000 €, the variable holding the computed tax contains a number roughly ten times smaller than it should — that is the **error**. The program then displays "Your estimated tax: €741.00" when the correct figure is €7,400.00. That discrepancy between what the spec required and what the user observes is the **failure**.

The chain matters for testing because testing operates by triggering failures and working backward. A bug that never executes never produces an error; an error that never propagates to observable output never produces a failure. A test that doesn't reach the buggy code, or that reaches it but doesn't check the output, will not detect the defect.

Why Testing Is Always Limited

Ziv's Law states that software development is unpredictable and that specifications will always be incomplete and inconsistent. This means you cannot fully know what the correct output is for all inputs even if you could run all of them.

Humphrey's Law observes that developers are systematically bad at finding defects in their own work. The mental model that produced the bug also shapes the test cases you write for

it — you tend to test what you expect to work and avoid the edge cases where your assumptions are fragile. This is why independent testing, peer review, and systematic techniques exist: to break the contamination between the perspective that wrote the code and the perspective that examines it.

The practical conclusion: the quality of a test suite is determined not by the number of test cases it contains but by the quality of the cases it selects. The rest of this unit is about how to select cases systematically rather than randomly.

---

2. Types of Testing

2.1 By Target: The Test Pyramid

The test pyramid, first described by Mike Cohn, has three main layers. The shape encodes a recommendation about how many tests of each kind you should have.

Unit tests sit at the base and should constitute roughly 70% of your total test count. A unit test exercises a single function, method, or class in complete isolation from its dependencies — if the function calls a database, the test substitutes a fake one. This isolation makes unit tests fast (milliseconds) and deterministic. They are written by developers, run on every commit, and are your first line of defence. When a unit test fails, the failure is localised: you know exactly which function broke.

Integration tests occupy the middle band, around 20%. Where unit tests fake dependencies, integration tests connect to real ones in a test environment. An integration test might verify that a function writing a payment record to a database produces the correct row with correct foreign keys. These tests are slower, potentially flaky, and harder to maintain — but they validate something unit tests cannot: that the pieces connect correctly at their seams.

End-to-end (E2E) tests sit at the top at around 10%. An E2E test drives the entire application from the outside — often through a browser — and verifies that a complete user journey produces the correct observable outcome. E2E tests are the slowest, the most brittle, and the most expensive to write and maintain. They are irreplaceable for scenarios that require the full system to behave correctly simultaneously (a checkout flow touching cart, payment gateway, inventory, and email notification).

Acceptance tests are sometimes treated as a fourth layer, testing the system from a stakeholder perspective: does this feature meet the business requirement as stated?

The **ice cream cone anti-pattern** is the inversion of the pyramid: very few unit tests, moderate integration tests, and a huge suite of E2E or manual tests. This is the natural state of legacy projects. The result is a test suite that takes hours to run (so developers stop running it), breaks for cosmetic reasons, and provides no useful fast feedback. Reversing

this pattern — writing unit tests for critical business logic and moving coverage downward — is one of the most valuable refactoring investments you can make in a legacy codebase.

2.2 By Knowledge: Black-Box, White-Box, Gray-Box

Black-box testing treats the system as a sealed box. The tester knows only the specification: what inputs are valid, what outputs are expected. Black-box tests are specification-faithful — they test what the system is supposed to do, not what the implementation happens to do. This also makes them language-agnostic. The risk is that a tester who only knows the specification may miss code paths that handle edge cases the specification forgot to mention.

White-box testing (clear-box or glass-box) treats the internals as fully visible. The tester reads the source code and designs tests to exercise specific structures: branches, error handlers, loops. White-box tests can reach into corners that a specification-only tester would never discover — but they risk being contaminated by the implementation's assumptions. If the code has a systematic logical error, white-box test cases derived from that code may repeat the same error in the expected results.

Gray-box testing sits between the two. The tester has partial knowledge — schema, architecture, algorithm — but not complete source code. Most practical integration and API testing is gray-box.

2.3 By Strategy: Usage-Based and Coverage-Based

Usage-based testing allocates test effort according to how the system is actually used. You identify the most frequent user journeys and most business-critical paths, and design your first tests around those. A bug in a path that 80% of users follow is far more damaging than a bug in a path that 2% follow. A streaming platform prioritises "browse → select → play" far above "edit subtitle preferences" because the former drives engagement and revenue.

Coverage-based testing fills the gaps left by usage-based testing. Once critical paths are covered, you use coverage metrics to identify code that has not yet been exercised and design tests specifically to reach it.

The critical insight is their **order**: usage-based comes first, coverage-based comes second. Inverting this order can produce a suite that covers 90% of obscure error-handling code while leaving the main checkout flow untested. Sequence matters.

2.4 Special-Purpose Tests

Smoke tests are a small, fast subset that verify the system is basically alive after a deployment — the application starts, database connections are established, and critical endpoints respond. They are the first thing you run before investing time in the full suite.

Regression tests are run whenever a change is made to verify that previously working behaviour still works. Every time a bug is fixed, a new regression test should be added to

prevent that bug from returning.

High-availability and chaos testing deliberately introduce failures into a running system to verify graceful degradation. Netflix's Chaos Monkey terminates random virtual machine instances in production to ensure redundancy and failover mechanisms actually work. Many systems that claim to be fault-tolerant turn out not to be when the fault actually happens.

---

3. The Language of Testing: Cases, Runs, and Suites

3.1 Test Case

A test case is a **static specification** of how to test one particular behaviour. It exists before anyone runs it. The same test case can be executed many times by many people in many environments — each execution is a separate test run, but the specification remains the same.

A well-written test case has exactly three required parts.

The **prerequisites** describe the exact initial state required before the test begins. Precision is essential: "the application is running" is not adequate; "the application is running, the test database has been reset to the standard fixture, and there is exactly one contact with name 'Ana García' and phone '+34 600 000 001'" is adequate. This precision enables reproducibility.

The **steps** describe the actions to take: numbered, unambiguous, granular. Each step should describe a single user action. "Navigate to contacts, select Ana García, and delete her" is three actions and should be three steps.

The **expected result** describes what the system should do after the steps. "The contact is deleted correctly" is not an expected result — it is a hope. "The contacts list displays 'No contacts' and the Ana García entry is no longer visible" is an expected result, because it is specific enough to determine unambiguously whether the test passed or failed.

TC-001: Delete the only contact in the address book

Prerequisites: The application is running. The contact database contains exactly one contact: name "Ana García", phone "+34 600 000 001". The user is on the Contacts List screen.

Steps:

1. Tap the entry for "Ana García".
2. Tap the "Delete" button.
3. Tap "Confirm" in the confirmation dialog.

Expected result: The Contacts List screen displays "No contacts" and the "Ana García" entry is no longer visible.

3.2 Test Run

A test run is a **dynamic instance** of a test case. Every test run records who performed it, when, in which environment (OS, browser version, device, application version), and what the actual result was compared to the expected result.

The same test case can pass in one environment and fail in another. TC-001 might pass on Firefox on Windows but fail on Safari on iOS, where the deletion appears to succeed but the entry reappears after navigating away. These are two different test runs of the same test case. This is why environment is recorded explicitly — without it, a failure report is nearly impossible to reproduce.

When a test run produces a failure, the accompanying **bug report** should contain: a descriptive title (not "login broken" but "Login fails silently for usernames containing an apostrophe"), severity, priority, exact reproduction steps, expected result, actual result, environment, and supporting evidence (screenshots, logs).

The distinction between **severity** and **priority** is critical. Severity is technical impact: how badly does the defect affect the system? Priority is business urgency: how quickly must it be fixed? These dimensions are independent.

A crash when importing CSV from Excel 2003 is high severity but potentially low priority (virtually no users use that format). A company logo appearing upside down is low severity (no data lost) but very high priority (every user sees it, brand damage, trivially easy to fix). Sorting your backlog purely by severity causes you to spend time on the Excel crash while the upside-down logo stays visible for days.

3.3 Test Suite

A test suite is a collection of test cases grouped for a specific purpose. Grouping matters because you rarely want to run every test after every change. If you change only the payment module, you want to run payment tests immediately, not the user profile tests.

Suites are grouped by feature, by level (all unit tests, all E2E), or by risk (a "critical path" suite). Good suite organisation enables targeted re-execution: after a patch to authentication, run the authentication suite immediately and queue the full regression for the nightly build.

---

4. The Testing Process: Plan, Execute, Analyse

4.1 Test Planning

Test planning answers the questions you need to answer before anyone writes a test case: What needs to be tested? What does not? When does testing end? Who is responsible? What resources are available? What constitutes "done"?

The most important output is the **exit criteria** — the specific, measurable conditions that must be met before the product is eligible for release. "Zero critical bugs and at least 95% of planned tests passing" is a well-formed exit criterion. "Testing is complete when QA is satisfied" is not — it is unmeasurable and creates political rather than technical release decisions.

Test planning also forces you to confront what you cannot test. If you have one week and a thousand test cases, prioritisation must happen during planning — not at execution time when pressure is highest.

4.2 Test Execution

During execution, the team runs test cases, records results, and files bug reports. The most common mistake is failing to record the results of tests that pass. Historical pass records are essential for two reasons.

First, regression detection requires knowing that something previously passed. Without a record of TC-045 passing in v2.1, you cannot tell whether its failure in v2.2 is a new regression or a pre-existing failure.

Second, trends reveal things individual results do not. A test that passes 90% of the time and fails 10% is a symptom of a flaky test or a race condition. You only discover this pattern with historical data.

4.3 Test Analysis and the Release Decision

Test analysis synthesises execution data into a quality assessment: how many critical defects remain open, are defect counts falling or rising, which modules have high defect density, are exit criteria met?

The **release decision** should be framed as risk management. Exit criteria will not always be perfectly met. The release decision is not "is the software perfect?" but "is the residual risk acceptable given the business context?" A medical device has a very different acceptable threshold than a marketing landing page. The job of test analysis is to make residual risk visible and quantified so stakeholders can make an informed decision rather than a hopeful one.

---

5. Designing Test Cases: Functional Techniques

5.1 Why Intuition Is Not Enough

Most people, when asked to write test cases for a function, write the obvious happy path first, add a couple of invalid inputs, and consider the job done. The triangle classification problem illustrates why this is insufficient.

Given a function that takes three side lengths and returns whether the triangle is equilateral, isosceles, scalene, or invalid, most people write four to six cases. Systematic analysis reveals ten to fifteen are needed, including: a side of zero (invalid), a negative side (invalid), the degenerate case where the sum of two sides exactly equals the third (spec needs to address this), all three permutations of isosceles triangles (a=b, b=c, a=c), and floating-point sides. The gaps in intuition-driven suites are systematic, not random — intuition gravitates toward normal cases and away from the boundaries and permutations where bugs hide.

The techniques in this section are systematic replacements for intuition. They do not require creativity; they require following a process.

5.2 Domain Partition (Equivalence Partitioning)

The core idea: the input space of any function can be divided into classes where every value in the class behaves the same way. Test one representative from each class and you have the same confidence as if you tested all values — at a fraction of the cost.

The process has three steps: identify all input variables; for each variable, enumerate distinct classes (valid inputs versus invalid inputs); pick one representative per class.

Consider the Spanish IRPF income tax function with three brackets: exempt (up to 13,000 €), first bracket (13,001–30,000 €, 24%), second bracket (>30,000 €, 37%). Income of zero or below is invalid. This gives four classes:

Class	Range	Representative
Invalid	≤ 0	−1
Exempt	1 – 13,000	6,500
First bracket	13,001 – 30,000	21,000
Second bracket	> 30,000	50,000

Testing −1, 6,500, 21,000, and 50,000 covers the entire input space with four test cases.

The triangle problem illustrates how domain partitioning handles multiple interacting variables. Each side has valid (positive) and invalid (zero or negative) classes. The three sides also interact: three equal sides = equilateral; exactly two equal = isosceles (three sub-cases: a=b, b=c, a=c); no equal sides = scalene; a side exceeding the sum of the other two = invalid by triangle inequality. Each is a distinct class requiring its own representative.

Note on GUIs: a form that only accepts positive numbers acts as a filter eliminating some invalid classes. But the moment an API provides direct access to the function, all invalid classes are back on the table.

5.3 Boundary Value Analysis

Domain partitioning identifies which classes to test; boundary value analysis identifies the most important values within and between classes. The motivation is empirical: defects cluster at boundaries. The most common single-character programming error in conditional logic is the confusion between `<` and `<=`. This error only manifests at the exact boundary value — all values strictly inside a class behave correctly regardless of which form was used.

Standard practice: test three points around each boundary — boundary − 1, the boundary itself, and boundary + 1. Always combine BVA with domain partition.

Applying this to IRPF with boundaries at 0, 13,000, and 30,000:

TC	Input	Technique	Expected result
TC01	−1	Domain (invalid)	Error: invalid income
TC02	0	Boundary (at 0)	Tax: 0
TC03	1	Boundary (0 + 1)	Tax: 0
TC04	6,500	Domain (exempt)	Tax: 0
TC05	12,999	Boundary (13,000 − 1)	Tax: 0
TC06	13,000	Boundary (at 13,000)	Tax: 0
TC07	13,001	Boundary (13,000 + 1)	Tax: 0.24
TC08	21,000	Domain (first bracket)	Tax: 1,920.00
TC09	29,999	Boundary (30,000 − 1)	Tax: 4,079.76
TC10	30,000	Boundary (at 30,000)	Tax: 4,080.00
TC11	30,001	Boundary (30,000 + 1)	Tax: 4,080.37
TC12	50,000	Domain (second bracket)	Tax: 11,480.00

To see what BVA catches that domain partition alone misses: suppose the implementation writes `income > 13000` instead of `income >= 13000`. TC04 (6,500) — both conditions evaluate the same; test passes. TC08 (21,000) — both evaluate the same; test passes. Only TC06 (13,000 exactly) distinguishes the two: `income > 13000` is false (exempt) but `income >= 13000` is true (taxable). Without BVA, this bug is invisible.

Key principle: if you cannot determine the expected result for a boundary case, the specification is incomplete. Raise the ambiguity before writing the test. A test with a guessed expected result is worse than no test — it encodes a wrong assumption as ground truth.

5.4 Decision Tables

Domain partition and BVA work when conditions are independent. Many real specifications involve conditions that interact: the output depends on the combination of several conditions simultaneously. Decision tables are designed for this situation.

Process: list the conditions, enumerate all 2^n combinations, fill expected results, then collapse combinations that produce identical results.

Consider an online store with three loyalty conditions (Gold membership, cart > 100 €, promotional code). Rules: all three = 30%; any two = 20%; any one = 10%; none = 0%.

With three conditions, $2^3 = 8$ combinations:

	R1	**R2**	**R3**	**R4**	**R5**	**R6**	**R7**	**R8**
Gold member	Y	Y	Y	Y	N	N	N	N
Cart > 100 €	Y	Y	N	N	Y	Y	N	N
Promo code	Y	N	Y	N	Y	N	Y	N
Discount	30%	20%	20%	10%	20%	10%	10%	0%

Rules 2, 3, 5 all produce 20% — collapse to "any two conditions = 20%". Rules 4, 6, 7 all produce 10% — collapse to "any one condition = 10%". The collapsed table has four rules, each representing one test case.

The decision table process also reveals specification ambiguities: if two combinations you expected to behave differently produce the same result, the spec may be overspecified. If a combination has no defined action, the spec has a gap. Either finding is valuable before a line of code is written.

---

6. Designing Test Cases: Structural Techniques

6.1 Why Structural Testing Complements Functional Testing

Functional tests verify the spec — but only the parts you thought to test. If the specification says "handle valid inputs correctly" and you write tests only for common inputs, you will not trigger defensive code for empty arrays or null pointers. Structural testing forces you to confront those paths: you look at the null check, see no functional test exercises its true branch, and add one.

Conversely, structural tests verify that code was executed — but say nothing about correctness. Whether the observed behaviour was evaluated against the right expected result remains a functional concern. Section 6.5 returns to this critical distinction.

6.2 Statement Coverage (C0)

Statement coverage asks: what percentage of executable statements in the code were executed by the test suite?

C0 = (statements executed) / (total executable statements) × 100%

Consider a simplified triangle classifier:

```
def classify_triangle(a, b, c):
    if a <= 0 or b <= 0 or c <= 0:            # line 2
        return "INVALID"                        # line 3
    if a == b and b == c:                       # line 4
        return "EQUILATERAL"                    # line 5
    if a == b or b == c or a == c:              # line 6
        return "ISOSCELES"                      # line 7
    if a + b > c and b + c > a and a + c > b:  # line 8
        return "SCALENE"                        # line 9
    return "INVALID"                            # line 10
```

Five return statements (lines 3, 5, 7, 9, 10). Each must be reached for 100% C0 — so you need at minimum five test cases: INVALID via first check, EQUILATERAL, ISOSCELES, SCALENE, and INVALID via triangle inequality.

The weakness of C0: it only requires each statement to be reached, not both branches of every condition. `if x > 0: return "positive"` achieves 100% C0 with a single test where x = 5. The false branch (x ≤ 0) is never exercised.

6.3 Branch Coverage (C1)

Branch coverage requires that every decision produces both a true and a false outcome in the test suite. A decision is any control flow bifurcation: if-statement, while condition, ternary operator, catch clause.

C1 = (branches exercised) / (total branches) × 100%

For the triangle classifier, four decisions × 2 branches = 8 total branches. 100% C1 requires exercising all eight — each condition true and false at least once. C1 implies C0 (if every branch is exercised, every statement is necessarily executed), but the converse is not true.

The practical industry standard is 70–80% branch coverage for production code, with higher thresholds (90–100%) for critical or safety-sensitive components. Measuring C1 and investigating untested branches reveals the gaps that matter most.

6.4 Path Coverage

Path coverage verifies that every distinct execution path through the function — every unique sequence of branches from entry to exit — is exercised.

```
function doSomething(x, y) {
    if (x > 0) { doA(); }  // Decision A
    if (y > 0) { doB(); }  // Decision B
}
```

Two decisions produce four paths: A-true/B-true, A-true/B-false, A-false/B-true, A-false/B-false. This growth is exponential:

Decisions	Distinct paths
2	4
5	32
10	1,024
20	1,048,576

100% path coverage is combinatorially infeasible for real functions. The practical value is that thinking in paths surfaces important test scenarios. For loops, the path perspective gives you the three boundary cases that matter: zero iterations (loop body never executes), one iteration, and many iterations. Covering these three catches the vast majority of loop-related defects.

Infeasible paths — paths that can never execute because conditions are logically mutually exclusive — should not be counted against coverage. A path requiring `x > 0` at line 5 and `x < 0` at line 10 (where x is never modified) is impossible regardless of any test you write.

6.5 Coverage vs Correctness

Coverage metrics measure how much of the code was executed. They measure nothing about whether the behaviour observed was correct.

Suppose you have 100% branch coverage for the IRPF calculator, but every test assertion is wrong — expected values computed using the same buggy formula the implementation uses. The coverage tool reports 100%. Every test passes. The software is broken; no metric tells you so.

Coverage gives confidence that you executed the code. It gives zero confidence about correctness. A test suite that executes every path with incorrect assertions is indistinguishable from a correct suite by any coverage tool.

Coverage is a lower bound: "this code was never executed" is a definitive gap. But "this code was executed" is only the beginning — whether the observation was evaluated against the right expected result remains a human responsibility, enforced by specification fidelity and — as Section 8 shows — mutation testing.

---

7. Test Automation

7.1 Why Automation Is Not Optional

Imagine a project requiring every new feature to trigger manual re-running of three hundred test cases. At twenty test cases, manual regression is manageable. At five hundred cases, it takes twelve hours. At a thousand cases, it takes days. No team executes a suite that takes days to run. They run a subset, accept higher regression risk, and periodically discover bugs users found that the team could have caught weeks earlier.

This is the **manual regression ceiling**: the empirical limit at which manual regression cost outpaces any team's capacity. Automation removes this ceiling. Five hundred automated tests run in seconds. The consequence: run the complete suite on every commit, pull request, deployment, and hotfix. Regressions are detected minutes after introduction rather than days.

Automation also delivers: **consistency** (no tired tester skipping steps), **documentation** (well-named tests are executable specifications — a new developer can read the suite to understand expected behaviour), and **safety** (confidence to refactor aggressively, knowing tests will catch anything broken).

7.2 Properties of Good Automated Tests

Self-checking means the test produces a binary result — pass or fail — without human interpretation. A test that logs output and requires a human to decide if it looks right is not self-checking.

```
// BAD: a human must inspect the output
it('calculates tax for income in the first bracket', () => {
    const result = getIRPF(21000);
    console.log('Tax result:', result);  // who decides if this is
correct?
});

// GOOD: the framework decides pass/fail
it('calculates tax for income in the first bracket', () => {
    const result = getIRPF(21000);
    expect(result).to.be.closeTo(1920, 0.01);
});
```

Repeatable means the test produces the same result every time, regardless of when or how many times it has run. Tests depending on current time, date, random numbers, or shared global state are **flaky tests** — intermittently failing for reasons unrelated to the code. Flaky tests train teams to ignore failures. When a red build is normal, a real failure blends into noise. Flakiness should be treated as a bug with the same priority as a production defect.

Simple means each test verifies one specific behaviour and has exactly one reason to fail. A broad test with ten assertions gives no guidance about which one failed or why.

```
// BAD: broad test — multiple reasons to fail
it('processes an order correctly', () => {
    const order = createOrder({ items: 3, total: 150, member: true });
    expect(order.discount).to.equal(20);
    expect(order.finalTotal).to.equal(120);
    expect(order.confirmationSent).to.be.true;
    expect(order.inventoryDeducted).to.be.true;
});

// GOOD: focused tests — each has one reason to fail
it('applies 20% discount when two conditions are met', () => {
    const order = createOrder({ total: 150, member: true, promo: false });
    expect(order.discount).to.equal(20);
});
it('sends confirmation after successful order', () => {
    const order = createOrder({ total: 50 });
    expect(order.confirmationSent).to.be.true;
});
```

Maintainable means tests are written against behaviour, not implementation details. A test that checks the value of a private variable is coupled to structure. Refactor the internal data representation and the test breaks — not because behaviour changed, but because the coupling was too tight.

7.3 Test Automation Frameworks

Framework	Language	Assertion style
Mocha + Chai	JavaScript	`expect().to.equal()`
Jest	JavaScript	`expect().toBe()`
JUnit	Java	`assertEquals()`
pytest	Python	`assert` statement
NUnit	C# / .NET	`Assert.That()`

This course uses **Mocha** as test runner and **Chai** as assertion library. Mocha provides `describe()` for grouping and `it()` for individual tests. Chai provides `expect()` for assertions. Tests run by opening `test.html` in a browser. For floating-point calculations, use `closeTo(expected, delta)` instead of `equal()` to avoid failures from rounding: `expect(getIRPF(21000)).to.be.closeTo(1920, 0.01)`.

7.4 The AAA Pattern

Every well-structured automated test follows **Arrange-Act-Assert (AAA)**.

Arrange sets up the preconditions: instantiate objects, configure state. Keep it minimal — only setup directly required for this test.

Act performs the one action being tested. One action per test. Three method invocations in Act means you are writing three tests.

Assert verifies the outcome. Ideally one logical assertion per test.

```
describe('getIRPF', () => {
    it('taxes the first euro above the exempt threshold at 24%', () => {
        // Arrange
        const income = 13001;

        // Act
        const tax = getIRPF(income);

        // Assert
        expect(tax).to.be.closeTo(0.24, 0.01);
    });
});
```

For simple tests, the three sections collapse into one readable line: `expect(getIRPF(6500)).to.equal(0)`. The pattern guides structure, not verbosity.

When a Chai assertion fails, the error message is immediate: "AssertionError: expected 1820 to be close to 1920 ±0.01". Good test names compound this: "getIRPF › first bracket › taxes income of 21000 correctly" tells you exactly which scenario failed before opening a file.

7.5 From Test Cases to Code: A Complete Example

The most important discipline in test automation is the order: **design on paper first, then automate, then run.** This protects against **contaminated testing** — the failure mode where a tester designs cases by reading the implementation rather than the specification.

If you read code containing a bug and write tests based on your reading, your tests encode the buggy behaviour as the expected behaviour. Every test passes. Coverage is 100%. The bug is invisible. You have tested that the code behaves like the code, not that the code behaves like the specification.

Protection against contamination: design test cases from the specification before looking at the implementation. Write inputs and expected outputs on paper. Only then open the source file to write automation. When you run the tests, a failure means the implementation diverges from the specification — which is exactly what you want to discover.

Complete `tests.js` for the TAXES function:

```
const { expect } = require('chai');
const { getIRPF } = require('./taxes');

describe('getIRPF', () => {

    describe('D1 — invalid inputs (income ≤ 0)', () => {
        it('TC01 — rejects income of -1', () => {
            expect(() => getIRPF(-1)).to.throw();
        });
    });

    describe('D2 — exempt income (0 to 13,000)', () => {
        it('TC02 — returns 0 for income of 0', () => {
            expect(getIRPF(0)).to.equal(0);
        });
        it('TC03 — returns 0 for income of 1', () => {
            expect(getIRPF(1)).to.equal(0);
        });
        it('TC04 — returns 0 for income of 6,500 (representative)', () =>
{
            expect(getIRPF(6500)).to.equal(0);
        });
        it('TC05 — returns 0 for income of 12,999 (boundary-1)', () => {
            expect(getIRPF(12999)).to.equal(0);
        });
        it('TC06 — returns 0 for income of 13,000 (boundary)', () => {
            expect(getIRPF(13000)).to.equal(0);
        });
    });

    describe('D3 — first bracket (13,001 to 30,000)', () => {
        it('TC07 — taxes income of 13,001 (boundary+1)', () => {
            expect(getIRPF(13001)).to.be.closeTo(0.24, 0.01);
        });
        it('TC08 — taxes income of 21,000 (representative)', () => {
            expect(getIRPF(21000)).to.be.closeTo(1920, 0.01);
        });
        it('TC09 — taxes income of 29,999 (boundary-1)', () => {
            expect(getIRPF(29999)).to.be.closeTo(4079.76, 0.01);
        });
        it('TC10 — taxes income of 30,000 (boundary)', () => {
            expect(getIRPF(30000)).to.be.closeTo(4080, 0.01);
        });
    });

    describe('D4 — second bracket (> 30,000)', () => {
        it('TC11 — taxes income of 30,001 (boundary+1)', () => {
            expect(getIRPF(30001)).to.be.closeTo(4080.37, 0.01);
        });
```

```
        it('TC12 – taxes income of 50,000 (representative)', () => {
            expect(getIRPF(50000)).to.be.closeTo(11480, 0.01);
        });
    });
});
```

Each `it()` corresponds to one row in the paper test case table. The description says what is being tested. The assertion encodes the expected result. If TC07 fails, you know immediately that the boundary at 13,001 is not handled correctly.

---

8. Mutation Testing

8.1 The Problem Coverage Cannot Solve

Coverage measures execution, not correctness. A test suite can achieve 100% branch coverage while all assertions are wrong. But even a suite with correct assertions has a subtler problem: it may have correct assertions for the inputs it tests, while having no assertion that would detect plausible bugs in the code it has executed.

Consider a test suite for IRPF with correct expected values but no test case at exactly 13,000. If a developer changes `income >= 13000` to `income > 13000`, every test still passes — because none exercises the input 13,000 exactly. The mutation survived.

Mutation testing asks a different question: "Would the tests detect a plausible change to this code?"

8.2 How Mutation Testing Works

Three steps: (1) the mutation tool creates **mutants** — modified versions of the production code, each with exactly one small change; (2) for each mutant, the full test suite runs against the mutated code; (3) results are classified: if at least one test fails, the mutant is **killed** — the suite detected that change. If all tests pass, the mutant **survived** — the suite has a gap.

Mutation score = (killed mutants) / (total mutants) × 100%

Mutations are drawn from a catalogue of **mutation operators** — transformations that correspond to common programmer mistakes:

Operator type	Example mutation
Relational	`>=` → `>`, `<` → `<=`, `==` → `!=`
Arithmetic	`+` → `-`, `*` → `/`
Logical	`&&` → `||`, `!` removed
Constant	`13000` → `12000`

Operator type	Example mutation
Statement deletion	An entire `return` statement removed

In this course, practice mutation testing manually: open `script.js`, change one thing, refresh `test.html`, observe which tests fail. Automated tools — Stryker (JavaScript), PIT (Java), mutmut (Python) — apply hundreds of mutants automatically.

8.3 Interpreting Results

Mutant 1: `<` instead of `<=` at the first bracket boundary. Implementation changes from `income >= 13000` to `income > 13000`. TC07 (13,001) does not kill it: both conditions evaluate to true for 13,001. TC04 (6,500) does not kill it: both evaluate to false. Only TC06 (13,000 exactly) distinguishes the two: `>= 13000` is true (taxable) but `> 13000` is false (exempt). Lesson: BVA boundary tests kill specific mutation operators that representative-value tests cannot reach.

Mutant 2: `FIRST_TAX_LIMIT = 12000` instead of `13000`. TC06 (income = 13,000) kills it: with original limit 13,000 is exempt; with limit = 12,000 it is taxable, so TC06's expected result of 0 fails. TC04 (income = 6,500) does not kill it: 6,500 is below both 12,000 and 13,000, so both versions place it in exempt. Lesson: the boundary test and the representative test kill different mutants; you need both.

Mutant 3: the entire third tax bracket conditional deleted. Income above 30,000 no longer triggers the higher rate. TC12 (income = 50,000) kills this immediately: the expected result of 11,480 is wildly different from what the mutated function returns. A suite testing only D2 and D3 representatives would have no test above 30,000 and would not kill this mutant. Lesson: every domain class needs at least one representative.

When a mutant survives, add the test case that kills it. A surviving mutant is evidence of a genuine gap. The test you add becomes a permanent part of the regression suite.

---

9. Going Beyond the Obvious

9.1 The Tester's Mindset

Technical techniques give you a systematic approach to the space the specification describes. But some of the most damaging bugs live in the space the specification forgot, in interactions the designer didn't anticipate, in misuse patterns the user will reliably discover.

Effective testers cultivate a distinctive cognitive posture.

Audaciousness means trying things the specification doesn't describe. If the spec says strings up to 255 characters, what happens at 256? At 10,000? If authentication is required, what happens with a syntactically valid but cryptographically invalid token?

Imagination means constructing realistic misuse scenarios. Real users are not rational actors following a manual. They misread instructions, hit back at inopportune moments, and use the application in contexts the designer never considered.

Questioning means challenging stated assumptions. "The system will be used by authenticated corporate users on standard hardware." What happens when the corporate VPN disconnects mid-session? What if a contractor uses a personal device?

Rapid learning and sensemaking are meta-skills: quickly forming a model of how the system works, identifying the model's most uncertain regions, and directing testing effort there. Expert testers can sketch the high-risk areas of an unfamiliar system faster than domain experts.

9.2 Angry Paths and Delinquent Paths

Angry paths simulate users who are impatient, confused, or frustrated — but not malicious. They click a submit button twice because the first click seemed unresponsive. They hit back immediately after submitting a payment. They open the same application in two browser tabs and complete different actions in each. These are normal human errors, and systems that handle only the canonical happy path fail in front of real users constantly.

The checkout double-click is classic: a slow network delays the confirmation page; the user clicks again. Does the system create two orders? Charge the card twice? Insert two database rows? A happy-path suite never clicks twice.

Delinquent paths simulate deliberate misuse. A delinquent user enters `' OR 1=1 --` in the username field (SQL injection). They paste JavaScript into a comment field (XSS). They manually edit the URL from `/account/123` to `/account/124` (horizontal privilege escalation). They submit a form without the CSRF token that JavaScript would normally append.

Security testing is a specialised discipline, but every developer and tester should know the OWASP Top 10. Delinquent path testing is not paranoia; it is the baseline of responsible development practice.

9.3 One Test Per Bug Found

When a bug is discovered, the discipline of software quality adds one step before the fix: write a test that reproduces the failure.

The value is threefold. First, writing the failing test forces you to understand the bug with enough precision to specify it — vague bug reports become precise test cases. Second, watching the test fail before applying the fix confirms your test case actually reproduces the bug. A surprising number of "confirmed" bug fixes turn out to fix a slightly different code path than the one that caused the failure. Third, the test becomes a permanent fixture in the regression suite. As long as it exists, that specific bug cannot silently return.

The alternative — fixing without a test — leaves the gap in the suite open. The bug can be reintroduced by any future refactoring that touches the same code. Under schedule pressure, "I'll write the regression test later" almost always means "there will be no regression test."

9.4 Bugs Cluster

Roughly 80% of defects in a codebase come from roughly 20% of the modules. A module containing one bug is significantly more likely to contain additional bugs than a module containing none.

The practical implication: when you find a defect in a specific area, do not move on. Test around it. Look at adjacent functionality, related edge cases, similar conditions. If the payment processing module has a bug in currency conversion, examine all other currency-related paths in the same module. The programmer who introduced the currency conversion bug may have made the same class of mistake in adjacent code.

This clustering heuristic complements systematic techniques rather than replacing them. Use it to allocate additional exploratory effort after systematic techniques have generated your baseline suite, directing that effort toward areas where evidence of defects already exists.

Key Terms Glossary

Acceptance testing — testing from a stakeholder perspective to verify that a feature meets its business requirements.

Angry path — a test scenario simulating a user who makes mistakes or behaves impatiently but without malicious intent (e.g., double-clicking submit).

Black-box testing — testing based solely on the specification, without knowledge of the implementation.

Branch coverage (C1) — a structural metric measuring the percentage of all true/false outcomes of every decision exercised by the test suite.

Bug (fault, defect) — a mistake in the source code: an incorrect statement, condition, or value.

Chaos testing — deliberately introducing failures into a running system to verify resilience and failover behaviour.

Contaminated testing — the failure mode where test cases are designed by reading the implementation rather than the specification, causing tests to encode bugs as correct behaviour.

Coverage-based testing — a strategy that uses structural metrics to identify and fill gaps after usage-based tests have covered high-priority paths.

Decision table — a technique for designing test cases when multiple conditions interact; enumerates all combinations and collapses equivalent ones.

Delinquent path — a test scenario simulating deliberate misuse or attack: SQL injection, privilege escalation, XSS.

Domain partition (equivalence partitioning) — dividing the input space into classes where all values behave identically, then testing one representative from each class.

E2E test (end-to-end) — a test exercising a complete user journey through the full application stack.

Error — internal state corruption from a bug executing; visible in memory but not yet to the user.

Exit criteria — specific, measurable conditions that must be met before testing is complete and the product is eligible for release.

Failure — the externally observable symptom of an error; the moment system behaviour diverges from specification in a way the user can detect.

Flaky test — a test that passes or fails intermittently for reasons unrelated to the code under test.

Gray-box testing — testing with partial knowledge of the implementation (schema, architecture) but not full source code.

Ice cream cone anti-pattern — an inverted test pyramid: few unit tests, moderate integration tests, a large proportion of slow, fragile E2E or manual tests.

Integration test — a test exercising real interactions between components or between a component and its dependencies.

Killed mutant — a mutation that caused at least one test to fail; indicates the suite is sensitive to that type of change.

Manual regression ceiling — the empirical limit at which running all tests manually outpaces a team's capacity, making automation necessary.

Mutation operator — a rule that transforms production code in a small, systematic way (e.g., changing `>=` to `>`).

Mutation score — the percentage of mutants killed by the test suite; a higher score indicates a more thorough suite.

Mutation testing — evaluating test suite effectiveness by introducing small code changes (mutants) and checking whether tests detect them.

Path coverage — a structural metric measuring whether every distinct execution path through a function has been exercised.

Priority — the urgency with which a defect should be fixed, based on business impact; distinct from severity.

Regression test — a test run after a change to verify that previously correct behaviour still behaves correctly.

Severity — the technical impact of a defect (crash, data loss, cosmetic); distinct from priority.

Smoke test — a minimal test verifying the system is alive and critical paths functional after a deployment.

Statement coverage (C0) — a structural metric measuring the percentage of executable statements reached by the test suite.

Survived mutant — a mutation that did not cause any test to fail; indicates a gap in the test suite.

Test automation — using code to execute test cases, verify expected results, and report failures without human intervention.

Test case — a static specification consisting of prerequisites, numbered steps, and an expected result; defines how to test one specific behaviour.

Test pyramid — a model recommending many unit tests (70%), fewer integration tests (20%), and even fewer E2E tests (10%).

Test run — a dynamic instance of a test case; records who executed it, when, in which environment, and the actual result.

Test suite — a collection of test cases grouped by feature, level, or risk to enable targeted re-execution.

Unit test — a test exercising a single function or class in complete isolation from real dependencies.

Usage-based testing — a strategy allocating effort by frequency and criticality of use; applied before coverage-based testing.

White-box testing — testing designed from source code knowledge, aimed at exercising specific code paths.

Ziv's Law — the observation that software development is inherently unpredictable and specifications will always be incomplete and inconsistent.